



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机系统设计实验报告

PA 4 实验报告

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：关浩

2026 年 5 月 28 日

目录

一、 必答题	2
(一) 必答题: 分页机制与硬件中断	2
二、 文内思考题	5
(一) Q: 基地址位数	5
(二) Q: 物理地址的必要性	5
(三) Q: 为什么不用一级页表	6
(四) Q: 空指针真的是”空”的吗	6
(五) Q: 内核映射的作用	7
(六) Q: 灾难性的后果	8
三、 代码实现	10
(一) TODO: 在 NEMU 中实现分页机制	10
(二) TODO: 让用户程序运行在分页机制上	14
(三) TODO: 在分页机制上运行仙剑奇侠传	17
(四) CHECKPOINT: PA4 stage1	18
(五) TODO: 实现内核自陷	19
(六) TODO: 实现上下文切换	20
(七) TODO: 分时运行仙剑奇侠传和 hello 程序	22
(八) TODO: 优先级调度	24
(九) CHECKPOINT: PA4 stage2	24
(十) TODO: 添加时钟中断	25
(十一) CHECKPOINT: PA4 stage3	27
四、 实验环境说明	28
(一) 虚拟机配置	28
(二) IDE 配置	28
五、 总结	29

一、 必答题

(一) 必答题：分页机制与硬件中断

问：请结合代码，解释分页机制和硬件中断是如何支撑仙剑奇侠传和 `hello` 程序在我们的计算机系统（Nanos-lite, AM, NEMU）中分时运行的。

答：

PAL 和 `hello` 能在这套系统里分时运行，靠的是两件事配合起来：

- **分页机制：**负责让两个程序拥有各自的地址空间和内存现场，解决“多个程序如何同时存在”。
- **硬件中断：**负责让 CPU 周期性地从当前程序手里回到内核，让 Nanos-lite 有机会执行 `schedule()` 切换到另一个程序，解决“内核如何定期夺回控制权”。

下面结合代码分别看两个机制如何起到支撑作用。

分页机制：

`main.c` 中通过 `load_prog()` 加载用户程序，每次 `load_prog()` 都会为一个 PCB 准备独立的地址空间：

```
1 _protect(&pcb[i].as);
2 uintptr_t entry = loader(&pcb[i].as, filename);
3 pcb[i].tf = _umake(&pcb[i].as, stack, stack, (void *)entry, NULL,
    NULL);
```

三个步骤：

- `_protect(&pcb[i].as)` 为每个进程创建自己的页目录。它们可以使用相同的虚拟地址范围，但通过不同页表映射会到不同物理页。
- `loader(&pcb[i].as, filename)` 在加载 ELF 时，通过 `_map()` 建立虚拟地址到物理地址的映射。程序看到的是自己的虚拟地址，真正访问内存时由分页机制根据当前 CR3 所指向的页目录翻译成物理地址。
- `_umake()` 为每个进程构造初始的 trap frame，也就是第一次被调度时要恢复的 CPU 现场。这个现场里有 `eip`, `esp`, `cs`, `eflags` 等。当前实现里尤其重要的是：

```
1 tf->eflags = 0x2 | FL_IF;
```

置位 `FL_IF` 打开中断，使用户进程运行后能够响应时钟中断。

当进程真正被切换时，`schedule()` 会调用：

```
1 _switch(&current->as);
2 return current->tf;
```

——切换当前地址空间，让 CPU 后续地址翻译使用新进程的页目录，对应到写 CR3。所以在进程调度过程中与分页机制直接相关的部分就是：

调度到 PAL -> CR3 指向 PAL 的页目录 -> 虚拟地址按 PAL 的映射解释
 调度到 hello -> CR3 指向 hello 的页目录 -> 虚拟地址按 hello 的映射解释

这就是分页对 PAL 与 hello 程序分时运行所提供的支撑：**两个程序可以同时装入内存，各自拥有独立的虚拟地址空间，调度时只需要切换当前地址空间，就能让同一套虚拟地址在不同进程中对应不同物理内容。**

硬件中断：

之前没有硬件中断机制时，调度依赖系统调用来中断。也就是用户程序主动执行 `int $0x80` 内核才有机会进行调度。这样有一个致命问题：如果某个程序一直计算、死循环、或者恶意不系统调用，内核就失去控制权。现在有了硬件中断，这种情况会被避免。

以本 PA 最终实现的时钟中断机制为准，NEMU 的时钟设备周期性触发 `timer_intr()`，调用：

```
1 dev_raise_intr();
```

这个函数内做了一个简单的置位操作：

```
1 cpu.INTR = true;
```

相当于发出了中断请求。

在这之后，由于 NEMU 在每条指令执行结束后，都会在 `exec_wrapper()` 末尾检查：

```
1 if (cpu.INTR && cpu.IF) {
2     cpu.INTR = false;
3     raise_intr(TIMER_IRQ, cpu.eip);
4     update_eip();
5 }
```

这里体现了硬件中断的核心语义：

INTR=1 表示有外部中断请求

IF=1 表示 CPU 当前允许响应中断

两者同时成立，CPU 才响应时钟中断

响应时钟中断时，`raise_intr(32, cpu.eip)` 会保存当前 `eflags/cs/eip`（必须明确的是，trap frame 的实现也是支持硬件中断的另一层必要工作，它保存了中断之前的完整运行状态便于恢复），然后根据 IDT 跳转到 32 号中断入口。当前实现还在保存旧 `eflags` 后清 IF：

```
1 rtl_push(&cpu.eflags);
2 cpu.IF = 0;
```

这表示进入中断处理后暂时关中断，避免当前阶段发生嵌套中断，这是按照实验手册要求把实现保持在了 KISS 准则范围内。

在 NEMU 层，只能够得到信息“发生了 32 号中断”，AM 的 ASYE 层则负责把它翻译成更上层能理解的事件。

ASYE 初始化时，代码注册了 32 号中断入口：

```
1 idt[32] = GATE(STS_TG32, KSEL(SEG_KCODE), vectimer, DPL_KERN);
```

trap.S 里的 vectimer 压入中断号 32, 然后进入统一的 asm_trap:

```
1 .globl vectimer; vectimer: pushl $0; pushl $32; jmp asm_trap
2
3 asm_trap:
4     pushal
5
6     pushl %esp
7     call irq_handle
```

进入调用的 irq_handle() 函数中后, ASYE 根据 tf->irq 包装事件:

```
1 case 32: ev.event = _EVENT_IRQ_TIME; break;
```

于是, NEMU 的硬件中断号 32 被 AM 包装成 Nanos-lite 能理解的 _EVENT_IRQ_TIME. Nanos-lite 层就根据 event 类型直接分发到相应操作:

```
1 case _EVENT_IRQ_TIME:
2     return schedule(r);
```

——调用 schedule() 进行进程调度。

所以硬件中断(这里是时钟中断)机制支持的完整进程切换过程是(以 PAL 切换到 hello 为例):

```
PAL 正在运行
NEMU 时钟到来, cpu.INTR = true
CPU 指令边界发现 INTR && IF
CPU 进入 32 号中断
AM 包装成 _EVENT_IRQ_TIME
Nanos-lite 调用 schedule()
schedule() 保存 PAL 的 tf
schedule() 选择 hello 或继续选择 PAL
_switch() 切换地址空间
trap.S 恢复被选中进程的 tf
被选中的进程继续运行
```

下一次时钟中断到来时, 又重复上面的过程; 如果是 hello 切换到 PAL, 将两者角色互换即可, 同理。

二、 文内思考题

PA 在编程指导过程中提出的一些问题，单列出来进行系统的回答。

(一) Q: 基地址位数

问: i386 不是一个 32 位的处理器吗，为什么表项中的基地址信息只有 20 位，而不是 32 位？

答:

实验手册中说明了，i386 将物理内存划分为 4KB 为单位的页面，而 4KB 对齐的地址低 12 位必然是 0。

一个 32 位物理地址可以写成：

$$\text{物理地址} = \text{物理页框基地址} + \text{页内偏移}$$

其中页内偏移占 12 位，因为 $4\text{ KB} = 4096\text{ bytes} = 2^{12}\text{ bytes}$ 。所以一个 4KB 物理页的起始地址一定是低 12 位为 0 的（实际上含义是页内偏移量为 0）。既然低 12 位永远是 0，就没有必要把它们存在 PDE/PTE 里。i386 于是把表项的高 20 位用来保存页表或物理页框的基地址；低 12 位则拿来放标志位，包括 P、R/W、U/S、A、D 等。

所以，“表项基地址只有 20 位”不是地址缩水，而是利用了 4KB 对齐的分页性质，把低 12 位省下来作为控制位。

(二) Q: 物理地址的必要性

问: 手册上提到表项（包括 CR3）中的基地址都是物理地址，物理地址是必须的吗？能否使用虚拟地址？

答:

对 i386 来说，必须是物理地址，而不能填普通虚拟地址。

原因是，页表本身就是用来完成“虚拟/线性地址到物理地址转换”的数据结构。CPU 查页表时，需要先从 CR3 找到页目录，再读 PDE 找到页表，再读 PTE 找到最终物理页框。如果 CR3 或 PDE/PTE 里存的是虚拟地址，逻辑无法闭环自治：如果要翻译虚拟地址，需要先读页表；要读页表，又需要先翻译页表的虚拟地址；要翻译页表的虚拟地址，又需要先读页表……陷入死循环。

所以 i386 规定了：

- CR3 里保存页目录的物理基地址；
- 页目录项 PDE 里保存下一级页表的物理基地址；
- 页表项 PTE 里保存目标物理页框的物理基地址。

在指导手册中也强调了：页目录、页表放在内存中，CR3 用来保存页目录基地址，page walk 会从 CR3 开始一步步查表。而 page walk 的“起点”必须是硬件可以直接拿去访问内存的物理位置。

(三) Q: 为什么不用一级页表

问: 为什么不采用一级页表? 或者说采用一级页表会有什么缺点?

答:

如果 i386 采用一级页表, 并且仍然保持: 32 位的虚拟/线性地址空间、4KB 的页面大小、每个 PTE 4 字节。那么需要的页表项数量可以计算出来: $2^{32}/2^{12} = 2^{20}$ 个页面。再者, 每个表项大小为 4 字节, 所以一级页表大小是: $2^{20} * 4 = 4\text{ MB}$

——也就是说, 如果一个进程拥有完整的 4GB 虚拟地址空间, 就需要一张 4MB 的一级页表。这存在很多缺点:

1. **浪费内存。**大多数程序并不会真的使用完整 4GB 虚拟地址空间。它可能只用了代码段、数据段、堆、栈和少量映射区域, 但一级页表仍然必须为整个地址空间准备 2^{20} 个表项。在进程很多的情况下, 页表本身就会消耗大量内存。
2. **不适合稀疏地址空间。**现代程序的虚拟地址空间往往是稀疏的, 中间大片区域并无映射。而如果采用一级页表, 就必须为所有虚拟页都留表项, 哪怕绝大多数表项都是无效的。
3. **分配和管理不便。**4MB 的连续页表是一块不小的内存。相比之下, 二级页表是拆成了很多 4KB 的页面: 一个页目录 4KB, 每个二级页表也是 4KB。颗粒度小后, 操作系统可以方便地按需分配页表页, 而不用一次性就准备完整的 4MB。

(四) Q: 空指针真的是”空”的吗

问: 程序设计课上老师告诉你, 当一个指针变量的值等于 NULL 时, 代表空, 不指向任何东西。仔细想想, 真的是这样吗? 当程序对空指针解引用的时候, 计算机内部具体都做了些什么? 你对空指针的本质有什么新的认识?

答:

从 C 语言语义的层面来说, NULL 不是“没有值”, 而是赋予一个特殊的指针值。C 语言标准规定: 值为 0 的整型常量表达式, 或转换成 `void *` 的这种表达式, 叫 “null pointer constant”; 它转换成某种指针类型后得到 null pointer, 并保证它不等于任何对象或函数的指针。

在常见 i386 / x86 系统里, 空指针通常指向数值为 0 的虚拟地址 (0x00000000)。当程序访问这个指针的时候, 编译器大致会生成一次从地址 0x00000000 读内存的指令。硬件内部会发生类似以下的过程:

1. CPU 执行 load 指令, 要访问地址 0x00000000。
2. 在 flat mode 下, 分段基本不改变地址, 线性地址仍然是 0x00000000。
3. 如果开启分页, MMU 用 CR3 指向的页目录进行 page walk。
4. 操作系统通常不映射虚拟地址 0 附近的页面。
5. 那么 MMU 查到对应 PDE/PTE 的 Present 位为 0。
6. 使得 CPU 触发 page fault 异常。
7. OS 的异常处理程序检查错误原因, 发现是非法访问, 而非正常的换页需求, 于是终止进程。如 Linux 下通常会表现为 Segmentation fault。

因此，空指针并不是真正的“空/什么都没有”，它其实存有一个有具体表示的值，只是这个值按照 C 语言规则不指向任何合法对象；解引用空指针导致的崩溃通常是 OS 利用分页保护机制制造出来的安全效果。

从不同层面来理解可以是：

- 语言层：NULL 是一个特殊指针值，保证不等于任何有效对象/函数地址。
- 机器层：在常见 i386 系统上，通常编码为数值 0。
- OS 层/MMU 层：地址 0 通常被特意设为不可访问，用 page fault 捕获解引用错误。

(五) Q：内核映射的作用

问：在 `_protect()` 函数中创建虚拟地址空间的时候，有一处代码用于拷贝内核映射：

```
1 for (int i = 0; i < NR_PDE; i++) {  
2     updir[i] = kpdirs[i];  
3 }
```

尝试注释这处代码，重新编译并运行，你会看到发生了错误。请解释为什么会发生这个错误。

答：

注释后运行发生的错误：

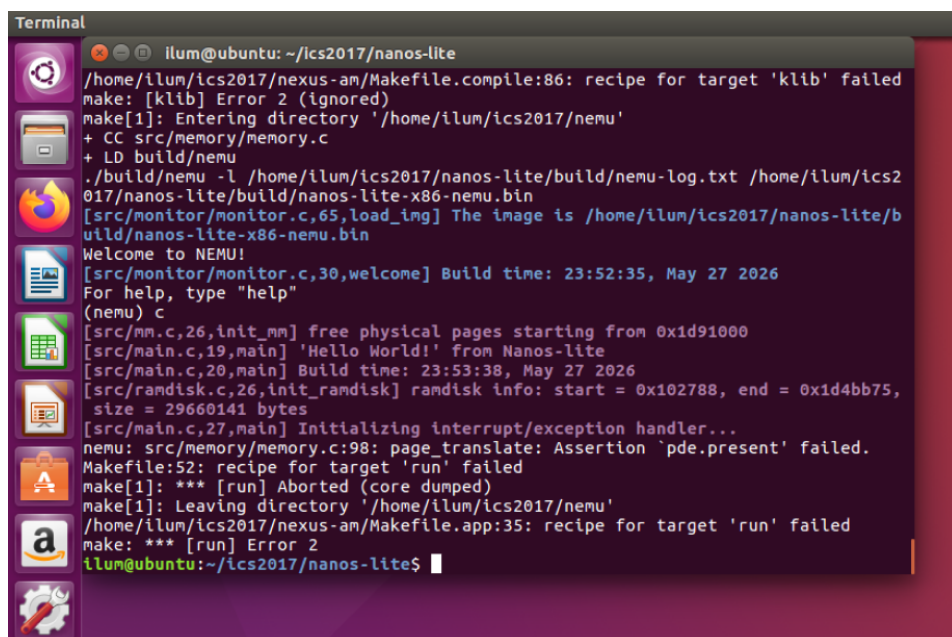


图 1：注释掉代码触发断言终止

```
nemu: src/memory/memory.c:98: page_translate: Assertion `pde.present' failed
```


这个错误的本质是：每个用户进程有自己的用户映射；但所有用户进程共享同一份内核映射。切换到用户进程页目录以后，CPU 仍然需要继续执行内核代码、访问内核数据和内核栈；如用户页目录里没有内核映射，进行内核相关操作时无法找到相应 PDE，程序无法继续运行。

发生的断言存在于之前实现的 `page_translate()` 函数中：

```
1 assert(pde.present);
```

此断言被触发，说明 NEMU 在做页级地址转换时，查到某个虚拟地址对应的 PDE 不存在，也就是一级页目录项的 `present` 位为 0。

在 `_pte_init()` 中，`kpgdirs` 是内核页目录，建立了内核使用的恒等映射。在 `pte.c` 中建立好：

```
1 kpgdirs[pdir_idx] = (uintptr_t)ptab | PTE_P;
```

而后通过：

```
1 set_cr3(kpgdirs);
```

让内核运行在这套页表上。再往后 `load_prog()` 会创建一个用户进程地址空间：

```
1 _protect(&pcb[i].as);
```

`_protect()` 里被注释掉的这段代码，其作用就是把原本内核映射复制进用户进程的新页目录 `updir` 中。这不是在映射用户程序本身，而是为了保证**当 CR3 切换到用户进程页目录后，内核地址仍然有效**。

如果注释掉，新建的 `updir` 里一开始几乎是空的。随后 `loader()` 通过 `_map()` 给用户程序入口附近建立映射，但内核的代码、数据、栈等地址怎么翻译，这部分信息没有被传递到用户进程。

此后 `load_prog()` 调用：

```
1 _switch(&pcb[i].as);
```

`_switch()` 将 CR3 切换到用户进程页目录 `updir`。然而 `_switch()` 之后 CPU 并不会瞬间跳进用户程序，它还要继续执行当前内核函数里的后续指令。这些后续指令的 EIP 仍然是内核代码地址，依赖 `_pte_init()` 建好的原本内核映射。但如上面所述，注释掉相应代码段以后，用户页目录里没有这些低地址内核 PDE。

所以，当下一次取指或访问内核栈/内核数据时，读取 `updir` 找内核的 PDE 发现找不到，触发 `assert(pde.present);` 的断言。

(六) Q：灾难性的后果

问：假设硬件把中断信息固定保存在内存地址 `0x1000` 的位置，AM 也总是从这里开始构造 trap frame。如果发生了中断嵌套，将会发生什么样的灾难性后果？这一灾难性的后果将会以什么样的形式表现出来？如果你觉得毫无头绪，你可以用纸笔模拟中断处理的过程。

答：

结论：

如果硬件把所有中断信息都固定保存在 `0x1000`，AM 也总是从 `0x1000` 开始构造 trap frame，那么一旦发生中断嵌套，后来的中断现场会覆盖前一个中断现场。

最直接的灾难是：第一次被中断的程序现场丢失，系统再也不知道该如何正确返回到它原来的执行位置。

这个后果会表现为：iret 返回到错误的地址、恢复错误的寄存器、调度器把不同进程的现场混为同一现场、不同进程的上下文互相污染，最终可能出现随机跳转、死循环、页错误、断言失败、panic 等异常。

详细分析：

下面结合代码逐步分析。

本 PA 的一个重点就在 trap frame 上，它保存了中断那一刻 CPU 所有的必要状态。处理中断事件的 irq_handle() 函数收到的就是一个 _RegSet *tf，它指向当前中断形成的现场。之后 irq_handle() 返回哪个 _RegSet *，trap.S 就把 %esp 切到相应的 trap frame，然后 popal 和 iret 从那里恢复现场。所以，trap frame 中存的 eip/cs/eflags/寄存器一旦不正确，CPU 就会恢复到错误的运行世界中。

现在假如硬件和 AM 把现场都写到固定地址 0x1000 中（也就 trap frame 起始地址），保存后现在内核开始处理这个时钟中断，进入 irq_handle() 函数。当前条件下，如果系统允许中断嵌套，那么在第一次中断还没处理完的时候（比如在内核还在执行 schedule() 或 do_event() 的中途），第二次中断又来了，由于规定硬件和 AM 固定地从 0x1000 开始保存现场，于是第二次中断也写到同一个地方，覆盖掉了上一个中断保存的 trap frame 的信息。于是当第一次中断返回时，系统以为自己恢复的是前一个中断前的状态，而实际上恢复的则是后一次的，因为 trap frame 已经被覆盖了。

根据中断的产生、状态保存以及恢复的整个代码过程，实际表现形式可能包括：

- iret 跳到错误的 eip，执行非法地址或错误代码。
- 用户程序突然从另一个程序的执行位置继续运行。
- 在现在多进程切换的例子下，PAL 的页表下恢复了 hello 的用户栈或代码地址，造成页表映射不匹配。
- eax/ebx/ecx/edx 被覆盖，系统调用号或参数变成随机值，触发 Unhandled syscall 或错误返回。
- irq 字段被覆盖，irq_handle() 误判事件类型，可能把时钟中断当成别的事件，或进入 _EVENT_ERROR。
- eflags 被覆盖，可能错误打开或关闭中断，导致中断风暴或再也不响应中断。
- 栈指针 esp 被恢复成错误值，后续 push/pop/call/ret 全部污染内存。
- 系统进入死循环、panic、assert 失败，或者表现为输出混乱、卡死、随机重启式行为等。

实际正确的实现是依靠栈的。中断现场必须保存在栈上，其核心原因就是栈天然支持嵌套，每次中断都会在原来的栈结构基础上压入新的 trap frame 结构，互不影响，且保持 LIFO 有序。所以每个 trap frame 都应该是排列在栈上的一段现场，而不是全局唯一的一块固定内存。

三、 代码实现

本节按照手册指导顺序，根据 TODO 任务和 CHECKPOINT 切分小节，逐步解析代码实现思路。

(一) TODO: 在 NEMU 中实现分页机制

根据指导手册建议的步骤，逐步添加分页机制相关的代码实现。

1. 添加 CR0/CR3 状态

在 reg.h 中把 CR0 cr0 和 CR3 cr3 加入 CPU_state，CR0/CR3/PDE/PTE 的结构定义已经存在于 mmu.h。而后，在 monitor.c 的 restart() 函数中按照要求初始化寄存器值：

```
1  /* [PA 4] 手册要求初始 CR0 为 0x60000011；PG 位仍为
   0，之后由内核写 CR0 开启分页。 */
2  cpu.cr0.val = 0x60000011;
3  cpu.cr3.val = 0;
```

这里 0x60000011 其 PG 位仍是 0，启动后默认不开分页；之后 AM 的 _pte_init() 会通过重新设置 CR0 和 CR3 的相关位来打开分页。

2. 实现操作 CR0/CR3 的指令

查阅 i386 后可知：

```
1 0f 20 /r    mov r32, CRn
2 0f 22 /r    mov CRn, r32
```

据此在 exec.c 的双字节 opcode 表中分别注册这两个指令。同时，针对这两个新的指令，在 decode.c 新增 decode_mov_cr() 函数，按照 ModR/M 字节解析控制寄存器编号和通用寄存器编号。

decode_mov_cr() 函数只接受 ModR/M.mod == 3，因为 mov CRx 只能和寄存器交换，不能用到内存操作数。若出现非寄存器形式，直接 assert()，符合指导书中要求的 KISS 原则。函数的代码如下：

```
1 static inline void decode_mov_cr(vaddr_t *eip, bool r2cr) {
2     ModR_M m;
3     m.val = instr_fetch(eip, 1);
4
5     assert(m.mod == 3);
6
7     if (r2cr) {
8         id_src->type = OP_TYPE_REG;
9         id_src->reg = m.R_M;
10        id_src->width = 4;
11        rtl_lr(&id_src->val, id_src->reg, id_src->width);
12
13        id_dest->type = OP_TYPE_REG;
14        id_dest->reg = m.reg;
15        id_dest->width = 4;
16    }
17    else {
```

```

18     id_src->type = OP_TYPE_REG;
19     id_src->reg = m.reg;
20     id_src->width = 4;
21
22     id_dest->type = OP_TYPE_REG;
23     id_dest->reg = m.R_M;
24     id_dest->width = 4;
25 }
26
27 #ifdef DEBUG
28     if (r2cr) {
29         snprintf(id_src->str, OP_STR_SIZE, "%%s",
30                 reg_name(id_src->reg, 4));
31         snprintf(id_dest->str, OP_STR_SIZE, "%%cr%d", id_dest->reg);
32     }
33     else {
34         snprintf(id_src->str, OP_STR_SIZE, "%%cr%d", id_src->reg);
35         snprintf(id_dest->str, OP_STR_SIZE, "%%s",
36                 reg_name(id_dest->reg, 4));
37     }
38 #endif
39 }

```

在 system.c 中实现了 CR0/CR3 的读写逻辑，用一个辅助函数对控制寄存器分发 I/O 方便后续添加其他的控制寄存器的支持：

```

1  /* [PA 4] 控制寄存器分发 */
2  static rtlreg_t* cr_reg(uint32_t reg) {
3      switch (reg) {
4          case 0: return &cpu.cr0.val;
5          case 3: return &cpu.cr3.val;
6          default:
7              assert(0);
8              return NULL;
9      }
10 }
11
12 // [PA 4] 从寄存器读出值写入 CR0/CR3
13 make_EHelper(mov_r2cr) {
14     *cr_reg(id_dest->reg) = id_src->val;
15
16     print_asm("movl %%s,%%cr%d", reg_name(id_src->reg, 4),
17             id_dest->reg);
18 }
19
20 // [PA 4] 把 CR0/CR3 的值写入对应寄存器
21 make_EHelper(mov_cr2r) {
22     rtlreg_t val = *cr_reg(id_src->reg);
23     operand_write(id_dest, &val);
24 }

```

```

23
24     print_asm("movl %%cr%d,%%s", id_src->reg, reg_name(id_dest->reg,
25         4));
26
27 #ifdef DIFF_TEST
28     diff_test_skip_qemu();
29 #endif
30 }

```

3. 修改 vaddr_read() / vaddr_write()

为了实现指导书中要求的“只有进入保护模式并开启分页机制之后才进行页级地址转换”语义，虚拟地址访问现在要检查 `cpu.cr0.paging`：

- 如果 PG 位为 0：保持原行为，虚拟地址直接当物理地址访问；
- 如果 PG 位为 1：调用 `page_translate()` 做页级地址转换，再访问物理地址。

这样，分页只在 `CR0.PG` 打开后才生效，满足语义。

此外，按照指导书要求，跨页情况暂挂不处理，普通情况下正常通过 `page_translate()` 访问即可。两个函数的代码如下：

```

1  uint32_t vaddr_read(vaddr_t addr, int len) {
2      if (!cpu.cr0.paging) {
3          return paddr_read(addr, len);
4      }
5
6      /* [PA 4]
7         跨虚拟页访问需要拆成两次翻译；本阶段先显式终止，等真正遇到再补
8         */
9      assert((addr & PAGE_MASK) + len <= PAGE_SIZE);
10
11     return paddr_read(page_translate(addr, false), len);
12 }
13
14 void vaddr_write(vaddr_t addr, int len, uint32_t data) {
15     if (!cpu.cr0.paging) {
16         paddr_write(addr, len, data);
17         return;
18     }
19
20     /* [PA 4] 同 vaddr_read(), 暂不处理跨页写入的数据拼接与拆分 */
21     assert((addr & PAGE_MASK) + len <= PAGE_SIZE);
22
23     paddr_write(page_translate(addr, true), len, data);
24 }

```

目前实现没有额外模拟完整保护机制，只用 PG 位作为入口条件。

4. 实现 page_translate()

实现新的函数 `page_translate()` 用于分页地址的转换。

基本的实现思路就是把 i386 的二级 page walk 翻译成几步位运算和物理内存访问：

```
vaddr[31:22] -> page directory index
vaddr[21:12] -> page table index
vaddr[11:0]  -> page offset
```

完整代码如下所示：

```
1 paddr_t page_translate(vaddr_t addr, bool is_write) {
2     uint32_t dir_idx = (addr >> 22) & 0x3ff;
3     uint32_t page_idx = (addr >> 12) & 0x3ff;
4     uint32_t offset = addr & PAGE_MASK;
5
6     paddr_t pdir_base = cpu.cr3.val & ~PAGE_MASK;
7     paddr_t pde_addr = pdir_base + dir_idx * sizeof(PDE);
8     PDE pde;
9     pde.val = paddr_read(pde_addr, 4);
10
11     /* 本阶段不实现 page fault
12      * 处理；遇到无效映射说明页表准备或翻译逻辑有误。 */
13     assert(pde.present);
14
15     if (!pde.accessed) {
16         pde.accessed = 1;
17         paddr_write(pde_addr, 4, pde.val);
18     }
19
20     paddr_t ptab_base = pde.page_frame << 12;
21     paddr_t pte_addr = ptab_base + page_idx * sizeof(PTE);
22     PTE pte;
23     pte.val = paddr_read(pte_addr, 4);
24     assert(pte.present);
25
26     if (!pte.accessed || (is_write && !pte.dirty)) {
27         pte.accessed = 1;
28         if (is_write) {
29             pte.dirty = 1;
30         }
31         paddr_write(pte_addr, 4, pte.val);
32     }
33
34     return (pte.page_frame << 12) | offset;
35 }
```

实验要求本阶段不实现 page fault 处理，因此对 PDE/PTE 的 `present` 位直接做 `assert()`。如果页表没准备好，NEMU 会尽早停止，而不是继续用错误地址乱跑，这对调试更友好。

此外，为了能够正确地进行 differential testing，按照 i386 分页机制补充了 `Accessed/Dirty`

位:

- 每次成功经过 PDE, 就设置 PDE 的 `accessed` 位;
- 每次成功经过 PTE, 就设置 PTE 的 `accessed` 位;
- 如果是写访问, 还设置 PTE 的 `dirty` 位。

函数整体的逻辑过程可以用如下的流程图概括:

```
CR3 -> 页目录物理基址
页目录基址 + dir_idx * 4 -> PDE
PDE.page_frame << 12 -> 页表物理基址
页表基址 + page_idx * 4 -> PTE
PTE.page_frame << 12 + offset -> 最终物理地址
```

注意函数中调用的必须都是物理地址处理相关的函数, 否则会形成依赖循环陷入逻辑死循环。

(二) TODO: 让用户程序运行在分页机制上

首先按照手册指导更改 `-Ttext` 参数和 `DEFAULT_ENTRY` 参数, 保证程序运行在虚拟地址上; 并且将原来 `main.c` 中用 `loader()` 函数直接加载用户程序换作经由 `load_prog()` 函数加载。

按照实验手册的阐述, 此时还需要实现 `_map()` 函数提供虚拟地址到物理地址的映射功能。

基本实现思路如下:

- `p->ptr` 取到用户进程页目录;
- 用 `PDX(va)` 找页目录项, 用 `PTX(va)` 找页表项;
- 如果目标 PDE 不存在, 就通过 `palloc_f()` 申请一页新的页表, 并清零;
- 把新页表的物理地址写入 PDE;
- 把 `pa | PTE_P` 写入对应 PTE, 建立 `va -> pa` 的一页映射。

`_map()` 的代码如下:

```
1 void _map(_Protect *p, void *va, void *pa) {
2     assert(p != NULL);
3     uintptr_t vaddr = (uintptr_t)va;
4     uintptr_t paddr = (uintptr_t)pa;
5     assert((vaddr & (PGSIZE - 1)) == 0);
6     assert((paddr & (PGSIZE - 1)) == 0);
7     assert(vaddr >= (uintptr_t)p->area.start && vaddr <
8             (uintptr_t)p->area.end);
9
10    PDE *pdir = (PDE *)p->ptr;
11    uint32_t pdir_idx = PDX(vaddr);
12    uint32_t ptab_idx = PTX(vaddr);
```

```

12
13 if ((pdir[pdir_idx] & PTE_P) == 0) {
14     PTE *ptab = (PTE *)palloc_f();
15     memset(ptab, 0, PGSIZE);
16
17     /*
18      * 页目录/页表在内核地址空间中恒等映射，因此这里写入的指针值同时也是物理页号
19      */
20     pdir[pdir_idx] = (uintptr_t)ptab | PTE_P;
21 }
22
23 PTE *ptab = (PTE *)PTE_ADDR(pdir[pdir_idx]);
24 assert((ptab[ptab_idx] & PTE_P) == 0);
25
26 /* 本阶段只需要建立映射，不实现权限保护；NEMU 侧只检查 present 位
27    */
28 ptab[ptab_idx] = paddr | PTE_P;
29 }

```

以上实现中，没有实现权限保护，只设置了 PTE_P，因为本阶段 NEMU 分页只检查 present 位，当前不需要实现完整的保护机制。

注意到，这里对一些异常情况进行检查。包括：

- p != NULL;
- va 和 pa 必须页对齐;
- va 必须落在用户地址空间 p->area 内;
- 不允许重复映射同一个 PTE。

所有异常全部直接 assert() 断言终止。

实现 _map() 后还需要对 loader() 做相应的修改，让用户程序确实被加载到虚拟地址中。loader() 不再能直接访问现在设置的 0x8048000 这个用户虚拟地址，而是应该先把数据放到真实物理页里，再为用户进程建立虚拟地址到物理页的映射。

新的 loader() 函数完整代码如下：

```

1 uintptr_t loader(_Protect *as, const char *filename) {
2     /* [PA 4] 强制通过地址空间参数建立映射。 */
3     assert(as != NULL);
4
5     int fd = fs_open(filename, 0, 0);
6     size_t size = fs_filesz(fd);
7
8     uintptr_t va = (uintptr_t)DEFAULT_ENTRY;
9     size_t offset = 0;
10    while (offset < size) {
11        void *pa = new_page();
12        size_t len = size - offset;
13        if (len > PGSIZE) {

```



```

14     len = PGSIZE;
15 }
16
17 /* [PA 4] loader
    运行在内核地址空间中，不能直接写用户虚拟地址，而是应该先填物理页再映射
    */
18 memset(pa, 0, PGSIZE);
19 _map(as, (void *)(va + offset), pa);
20 fs_read(fd, pa, len);
21 offset += len;
22 }
23
24 fs_close(fd);
25 return (uintptr_t)DEFAULT_ENTRY;
26 }

```

其流程可以概括为：

打开用户程序文件

|-> 获取文件大小

|-> 从设置的虚拟起始地址开始，按页处理

|- 每页调用 `new_page()` 申请一个物理页

`- 用 `memset()` 清空申请到的物理页，避免残留旧数据

|-> 调用 `_map(as, va, pa)` 把用户虚拟页映射到新物理页

`-> 用 `fs_read(fd, pa, len)` 把文件内容读入物理页

关键步骤就在于 `new_page()`, `memset()`, `_map()` 和 `fs_read()` 这几个函数的调用，它们构成了一个物理页从申请、初始化到映射给虚拟页面的完整处理流程。

以上代码实现完整后，重新 `make update` 并 `make run`，发现 NEMU 出现断言终止：

```

Terminal
ilum@ubuntu: ~/ics2017/nanos-lite
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/ilum/ics2017/nexus-am/libs/klib'
/home/ilum/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/ilum/ics2017/nemu'
./build/nemu -l /home/ilum/ics2017/nanos-lite/build/nemu-log.txt /home/ilum/ics2
017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/ilum/ics2017/nanos-lite/b
uild/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 23:52:35, May 27 2026
For help, type "help"
(nemu) c
[src/mm.c,26,init_mm] free physical pages starting from 0x1d91000
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 23:53:38, May 27 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x1027a8, end = 0x1d4bb95,
size = 29660141 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
nemu: src/memory/memory.c:50: vaddr_read: Assertion `(addr & PAGE_MASK) + len <=
PAGE_SIZE' failed.
Makefile:52: recipe for target 'run' failed
make[1]: *** [run] Aborted (core dumped)
make[1]: Leaving directory '/home/ilum/ics2017/nemu'
/home/ilum/ics2017/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Error 2

```

图 2: 仅实现分页无跨页支持的程序运行终止

这是之前在 `vaddr_read()` 中对于“读内存操作跨越虚拟页边界”情况直接 `assert()` 的写导致的。在这里，运行 `dummy` 可能是取指、读描述符或其它非对齐访问刚好跨页，于是触发了这个断言。所以还是需要实现读和写的跨页。

跨页读和写的逻辑是，将跨页访问拆成当前页剩余字节、下一页开头字节两段，读操作按小端序拼回 `uint32_t`、写操作按小端序拆成低地址页和高地址页分两次写入。

在 `vaddr_read()` 和 `vaddr_write()` 到达跨页情况的地方，都添加：

```
1 int left = PAGE_SIZE - (addr & PAGE_MASK);
2 int right = len - left;
```

然后在 `vaddr_read()` 按照上面逻辑实现跨页读：

```
1 low = paddr_read(page_translate(addr, false), left);
2 high = paddr_read(page_translate(addr + left, false), right);
3 return low | (high << (left << 3));
```

在 `vaddr_write()` 按照上面逻辑实现跨页写：

```
1 low_mask = (1u << (left << 3)) - 1;
2 paddr_write(page_translate(addr, true), left, data & low_mask);
3 paddr_write(page_translate(addr + left, true), right, data >> (left
    << 3));
```

通过上面的修改，再重新 `make update` 并 `make run`，运行结果如下：

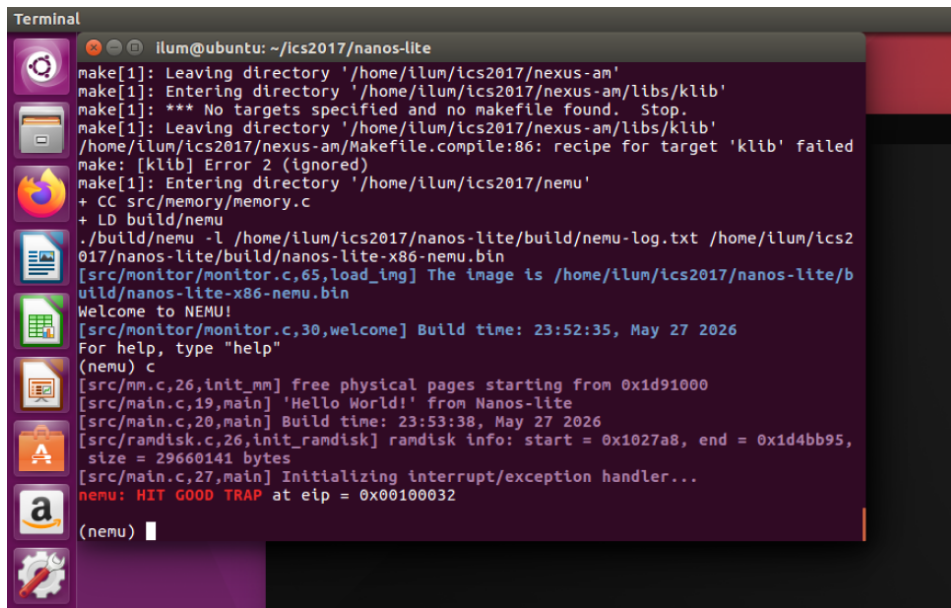


图 3: 跨页读写机制实现后运行成功

可以看到，和指导手册预期一样到达了 `GOOD TRAP`，说明以上分页机制和 `va->pa` 映射实现是正确的。

(三) TODO: 在分页机制上运行仙剑奇侠传

按照手册指示，本节还需要在 `mm_brk()` 函数中把新申请的堆区映射到虚拟地址空间中。补充在手册中标识为“TODO”区域的代码：

```

1 uintptr_t va = PGROUNDUP(current->max_brk);
2 uintptr_t end = PGROUNDUP(new_brk);
3
4 for (; va < end; va += PGSIZE) {
5     void *pa = new_page();
6
7     memset(pa, 0, PGSIZE);
8     _map(&current->as, (void *)va, pa);
9 }

```

这样对齐是因为当前的 `_map()` 要求虚拟地址和物理地址都按页对齐；而 `brk` 传进来的 `new_brk` 不保证页对齐。映射区间语义仍然是手册里的 `[current->max_brk, new_brk)`，只是实际调用 `_map()` 时要按页粒度覆盖它需要的新页。

实现了堆区映射后，再次 `make update` 并 `make run`，运行结果如下：

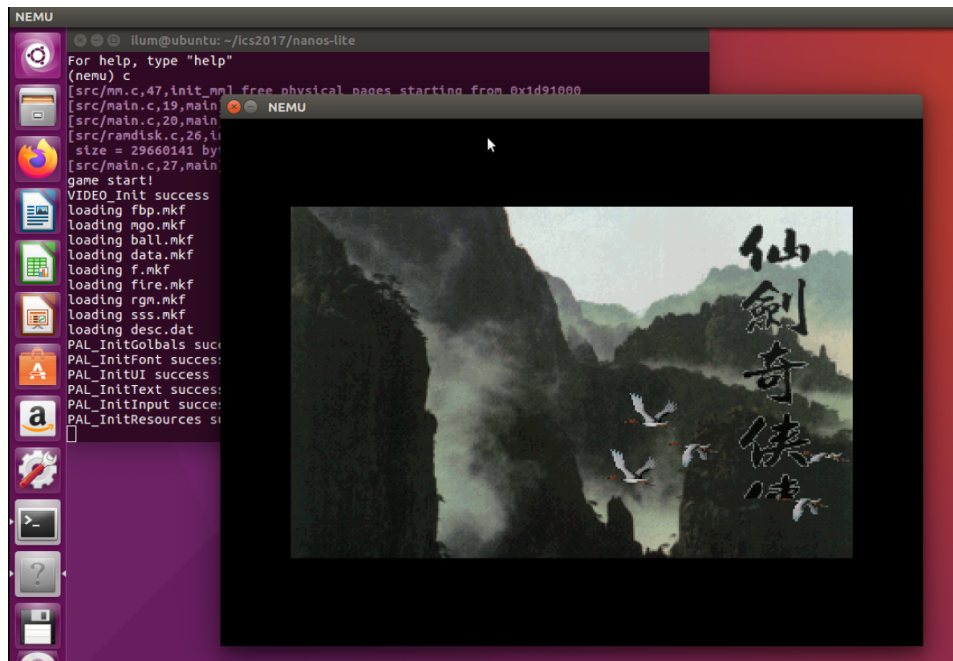


图 4: 在分页机制上运行 PAL

游戏能够成功在虚拟地址上运行，上述代码实现正确。

(四) CHECKPOINT: PA4 stage1

——至此 PA4 stage1 完成。

手动 git 本地记录如下：

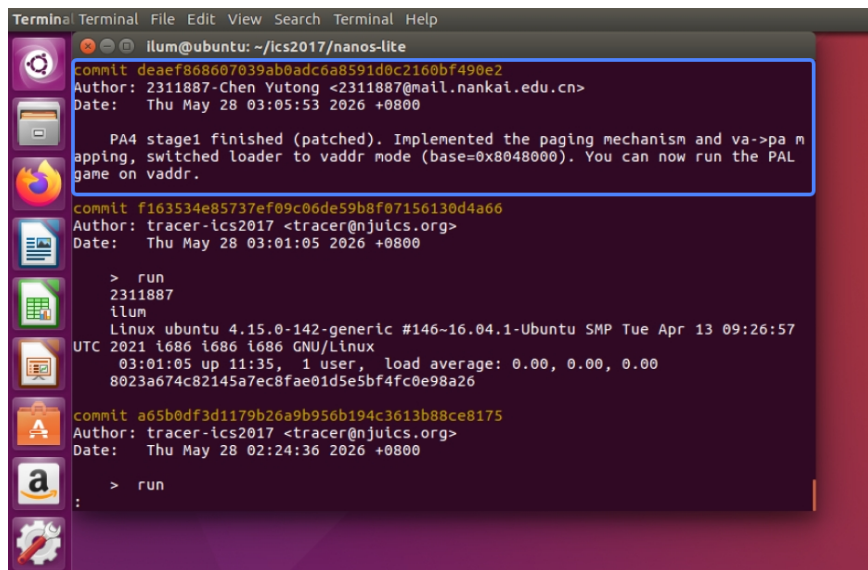


图 5: 手动 git 本地记录

(五) TODO: 实现内核自陷

首先, 在 ASYE 的 trap 入口里新增 vectrap, 它和 vecsys 一样进入公共 asm_trap, 但压入的 irq id 是 0x81. 这样, 按照 i386 规定, _trap() 执行 int \$0x81 后, irq_handle() 就能够从 trap frame 里读到 tf->irq == 0x81, 识别为内核自陷。添加代码行:

```
1 .globl vectrap; vectrap: pushl $0; pushl $0x81; jmp asm_trap
```

下一步在 ASYE 初始化和事件包装中识别内核自陷。先在 irq_handle() 中识别 0x81 判断为 _EVENT_TRAP:

```
1 switch (tf->irq) {
2     case 0x80: ev.event = _EVENT_SYSCALL; break;
3     /* [PA 4] x86-nemu 约定 int $0x81 表示内核自陷。 */
4     case 0x81: ev.event = _EVENT_TRAP; break;
5     default: ev.event = _EVENT_ERROR; break;
6 }
```

而后再在 _asye_init() 初始化 idt[0x81] 表项为 vectrap:

```
1 // ----- system call -----
2 idt[0x80] = GATE(STS_TG32, KSEL(SEG_KCODE), vecsys, DPL_USER);
3 /* [PA 4] 为内核自陷保留 0x81 号中断向量。 */
4 idt[0x81] = GATE(STS_TG32, KSEL(SEG_KCODE), vectrap, DPL_KERN);
```

然后还需要在 _trap() 中通过执行 int 0x81 进入统一的陷入处理流程:

```
1 void _trap() {
2     /* [PA 4] 通过保留向量进入统一的陷入处理流程。 */
3     asm volatile("int $0x81");
4 }
```

注意这里让 0x81 的 IDT 表项使用 DPL_KERN, 因为手册这一节明确称它为“内核自陷”, 而不是用户态系统调用入口; 0x80 的 syscall 仍保持使用 DPL_USER.

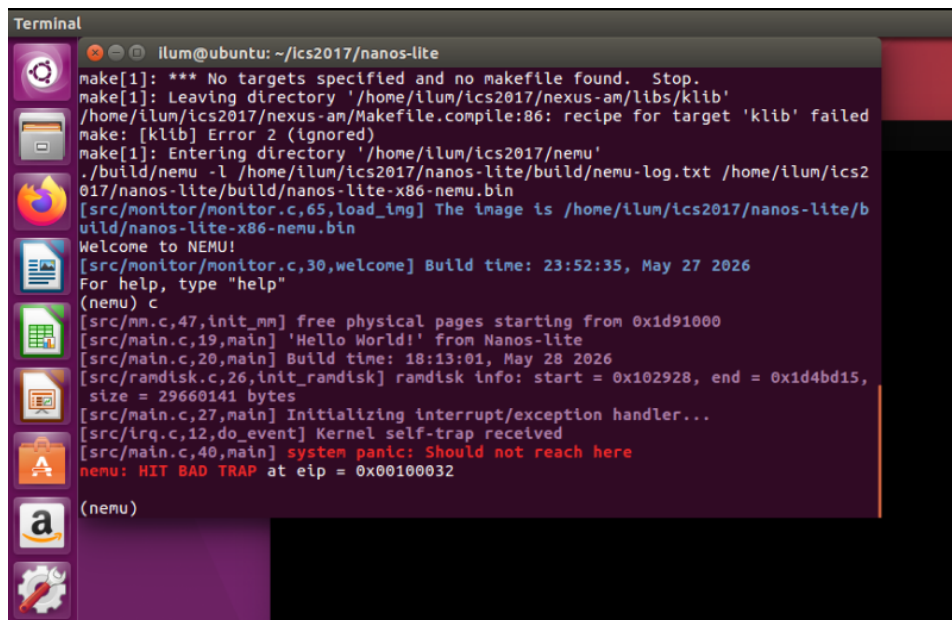
最后在 `irq.c` 的 `do_event()` 函数中添加 `_EVENT_TRAP` 的分支即可：

```

1 switch (e.event) {
2     case _EVENT_SYSCALL:
3         /* [PA 3] 系统调用事件交给 syscall
4            分发器处理，并沿用当前现场返回 */
5         return do_syscall(r);
6     case _EVENT_TRAP:
7         /* [PA 4] 本阶段只验证内核自陷事件能被正确送达。 */
8         Log("Kernel self-trap received");
9         return r;
10    default: panic("Unhandled event ID = %d", e.event);
11 }

```

以上代码实现完毕后，重新 `make update` 并 `make run`，结果如下所示：



```

Terminal
illum@ubuntu: ~/ics2017/nanos-lite
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/illum/ics2017/nexus-am/libs/klib'
/home/illum/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/illum/ics2017/nemu'
./build/nemu -l /home/illum/ics2017/nanos-lite/build/nemu-log.txt /home/illum/ics2
017/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/illum/ics2017/nanos-lite/b
uild/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 23:52:35, May 27 2026
For help, type "help"
(nemu) c
[src/mm.c,47,init_mm] free physical pages starting from 0xid91000
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 18:13:01, May 28 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102928, end = 0x1d4bd15,
size = 29660141 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/irq.c,12,do_event] Kernel self-trap received
[src/main.c,40,main] system panic: Should not reach here
nemu: HIT BAD TRAP at eip = 0x00100032
(nemu)

```

图 6: 仅实现内核自陷触发 `panic`

触发了 `main.c` 最后的 `panic`，与手册描述一致，实现正确。

(六) TODO: 实现上下文切换

按照实验手册建议步骤按步补充代码，实现完整的内核自陷触发上下文切换机制。

1. 实现 `_umake()` 函数

`_umake()` 的目标功能是人工构造一个陷阱帧，让 `asm_trap` 未来可以像恢复普通中断现场一样恢复用户程序。函数代码如下：

```

1 _RegSet *_umake(_Protect *p, _Area ustack, _Area kstack, void
2     *entry, char *const argv[], char *const envp[]) {
3     /* ... 预留接口略 ... */
4     uintptr_t *sp = (uintptr_t *)ustack.end;

```

```

5
6  /* 为 _start(argc, argv, envp) 准备初始栈帧；返回地址不会被使用。
   */
7  *--sp = 0; // envp, 环境变量指针
8  *--sp = 0; // argv, 参数数组指针
9  *--sp = 0; // argc, 参数个数
10 *--sp = 0; // 返回地址, 不会被实际使用
11
12  _RegSet *tf = (_RegSet *)((uintptr_t)sp - sizeof(_RegSet));
13  assert((uintptr_t)tf >= (uintptr_t)ustack.start);
14  memset(tf, 0, sizeof(*tf));
15
16  /* iret 将从这个人工陷阱帧恢复到用户程序入口。 */
17  tf->eip = (uintptr_t)entry;
18  /* 手册指导要求 cs 为 8; EFLAGS 的 bit 1 在 i386 中必须为 1。 */
19  tf->cs = KSEL(SEG_KCODE);
20  tf->eflags = 0x2;
21  tf->esp = (uintptr_t)sp;
22
23  return tf;
24 }

```

思路是：

- 从 ustack.end 向低地址构造 _start(argc, argv, envp) 看到的初始栈帧，并预留返回地址、argc、argv、envp，当前都置 0/NULL。
- 在这个 _start 栈帧下方放置人工 _RegSet。
- 设置 tf->eip = entry，让 iret 后进入用户程序入口。
- 根据手册指导设置 trapframe 的状态位，具体略。

2. 实现 schedule() 函数

通过 schedule() 函数实现对进程的调度。根据实验手册要求，目前 schedule() 只需要总是切换到第一个用户进程即可，即 pcb[0]：

```

1  _RegSet* schedule(_RegSet *prev) {
2      assert(nr_proc > 0);
3
4      if (current != NULL) {
5          /* [PA 4] 保存当前进程这次陷入形成的现场位置。 */
6          current->tf = prev;
7      }
8
9      /* [PA 4] 本阶段只调度第一个用户进程。 */
10     current = &pcb[0];
11     _switch(&current->as);
12     return current->tf;
13 }

```


其中调用 `_switch(¤t->as)` 切换到选定进程的页目录。函数返回 `current->tf`，递交给 ASYE 恢复现场。

3. 修改 `asm_trap()` 的实现

修改 ASYE 的 trap 逻辑，让它收到 `irq_handle()` 的返回值后，把 `%esp` 切到返回的陷阱帧，再 `popal/iret`，也就是手册中说的“先将栈顶指针切换到新进程的陷阱帧，然后才根据陷阱帧的内容恢复现场”，对应上下文切换本质操作。修改后的 `asm_trap()`：

```
1 asm_trap:
2     pushal
3
4     pushl %esp
5     call irq_handle
6
7     movl %eax, %esp
8
9     popal
10    addl $8, %esp
11
12    iret
```

上述代码实现完毕后，重新 `make update` 并 `make run`，运行结果如下：

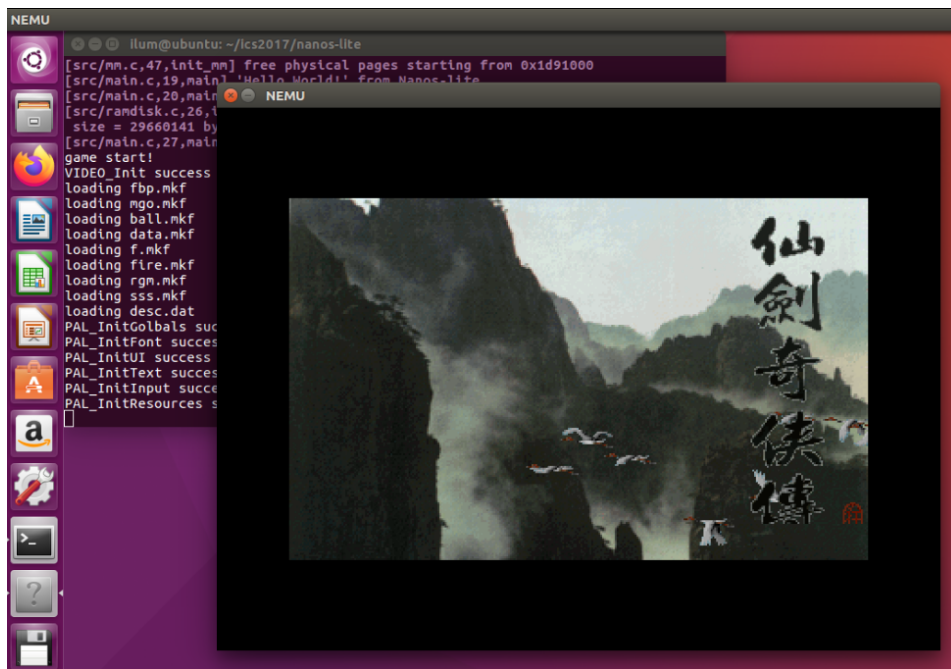


图 7: 实现上下文切换后成功运行 PAL

能够正常运行，说明 Nanos-lite 已可通过内核自陷触发上下文切换的方式运行仙剑奇侠传，代码实现正确。

(七) TODO: 分时运行仙剑奇侠传和 hello 程序

此步目标是让仙剑奇侠传和 hello 程序分时运行。

按照手册指导在 main.c 中添加了 /bin/hello 程序的加载，将在合适时机触发调度，让两个程序“同时”运行。

此后修改 schedule() 的调度策略，遵循手册建议的最简单调度方案，即“在已经加载的 PCB 之间轮流切换”。具体而言，仍然先保存当前进程的现场 `current->tf = prev`，但不再固定调度 `pcb[0]` 而是改为在 `pcb[0] ... pcb[nr_proc - 1]` 之间简单轮转，每次选中新进程后调用 `_switch(¤t->as)`。实际代码：

```
1 if (current == NULL || current == &pcb[nr_proc - 1]) {
2     current = &pcb[0];
3 }
4 else {
5     current++;
6 }
```

第一次 `_trap()` 进入时有 `current == NULL`，因此仍然从 PAL 开始；之后 PAL 和 hello 的系统调用会轮流触发切换。

合适的调度时机应是“系统调用处理完之后”，所以将 `_EVENT_SYSCALL` 在 `do_syscall(r)` 操作之后调用 `schedule(r)`，和 `_EVENT_TRAP` 一样每次都必须进行调度。代码更改：

```
1 switch (e.event) {
2     case _EVENT_SYSCALL:
3         /* [PA 4] 系统调用处理完后触发一次调度，实现进程分时运行。 */
4         do_syscall(r);
5         return schedule(r);
6 }
```

上述代码实现完毕后，重新 `make update` 并 `make run`，运行结果如下：

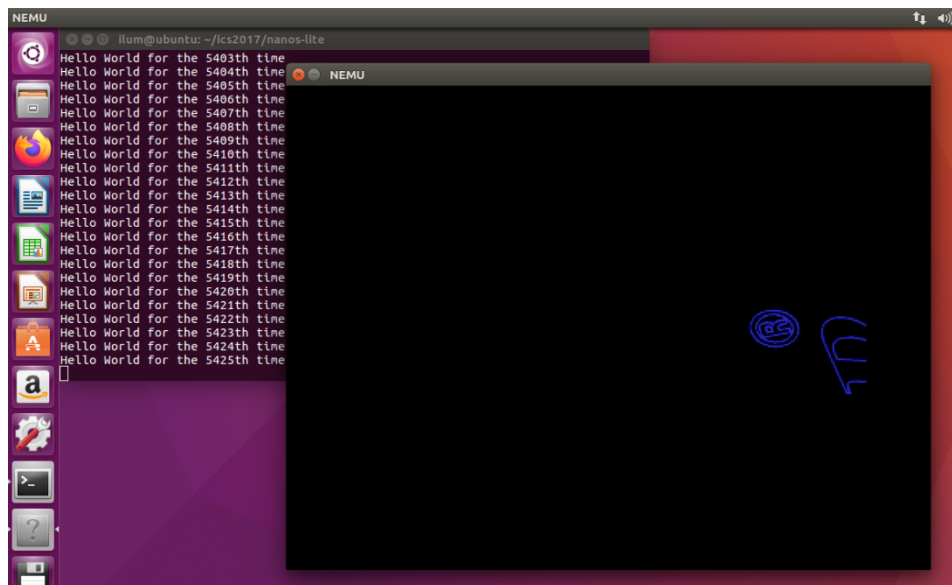


图 8: 同时运行 PAL 和 hello 程序

可以看到，确实是在 PAL 运行的同时也在输出 Hello World，不过 PAL 运行非常缓慢，可能因为 NEMU 虚拟出来的主机性能差，hello 程序频繁占据主要的输出串口导致 PAL 无法获得足够的运行资源。不过，既然两者能够同时运行，就说明上面的代码实现是正确的。

(八) TODO: 优先级调度

上面实验发现实现分时运行后 PAL 运行速度显著下降, 为了保持一定的运行效能, 对 `schedule()` 函数中的调度方式进行优化, 调整二者调度的频率比例, 使得仙剑奇侠传调度若干次才让 `hello` 程序调度 1 次。代码修改如下:

```
1 /* [PA 4] 优先调度 PAL 多次, 只偶尔调度 hello 确认它仍在运行。 */
2 else if (pal_schedule_count < PAL_SCHEDULE_QUOTA) {
3     current = &pcb[0];
4     pal_schedule_count ++;
5 }
6 else {
7     current = &pcb[1];
8     pal_schedule_count = 0;
9 }
```

其中调度配额 `#define PAL_SCHEDULE_QUOTA 1000` 设置为 1000.

修改后再 `make update` 并 `make run`, 运行结果如下:

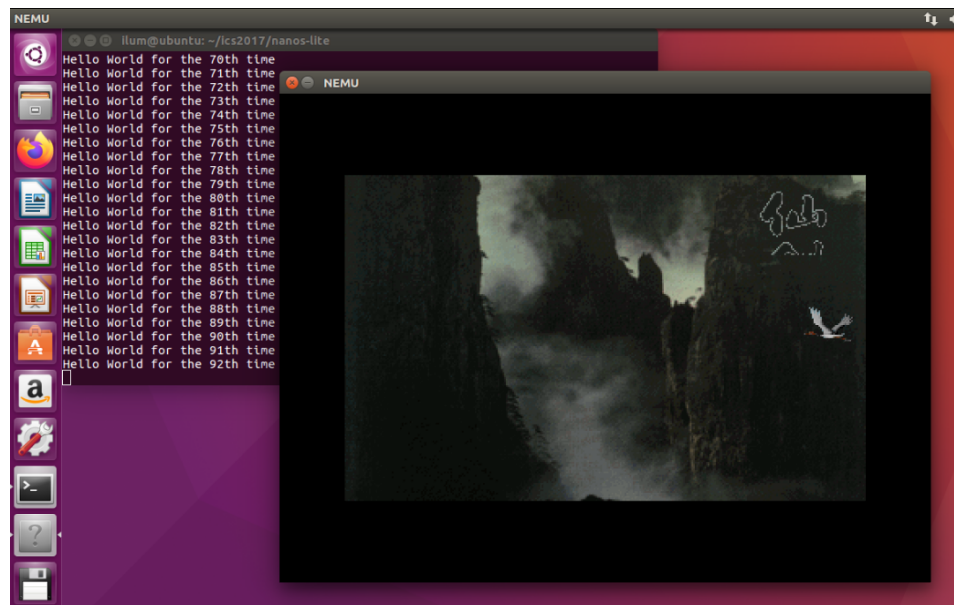


图 9: 更有效地同时运行 PAL 和 `hello` 程序

`hello` 程序只是很偶尔地进行输出, 而 PAL 基本恢复到了原先的运行速度。说明对于 `schedule()` 调度策略的修改是有效的。

(九) CHECKPOINT: PA4 stage2

——至此 *PA4 stage2* 完成。

手动 `git` 本地记录如下:

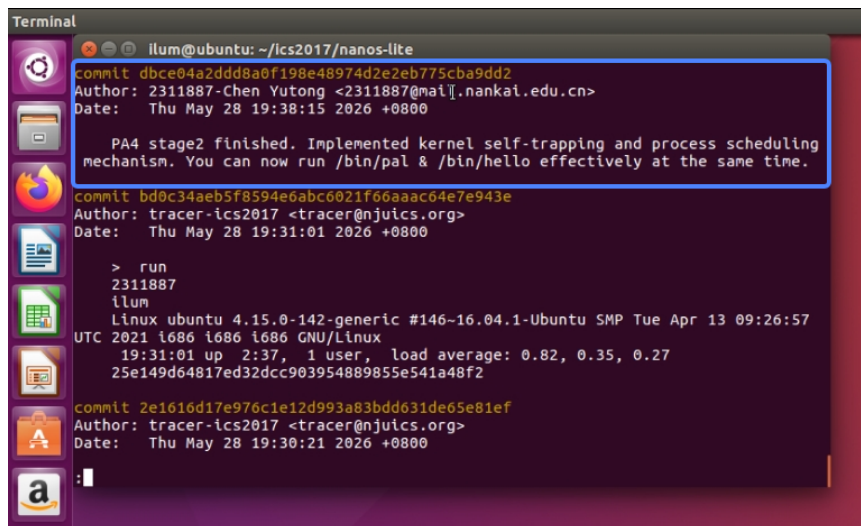


图 10: 手动 git 本地记录

(十) TODO: 添加时钟中断

根据讲义内容，添加相应代码来实现真正的分时多任务。

实现分为硬件/软件两方面，下面是主要思路。

硬件方面：时钟设备不直接打断 CPU，而是通过 `dev_raise_intr()` 置位 `cpu.INTR`；CPU 在每条指令执行结束后检查 `INTR && IF`，若允许响应中断，就清掉 `INTR`，调用 `raise_intr(32, cpu.eip)` 进入中断入口。进入中断时先保存旧 `EFLAGS`，再清除 `IF` 位，让中断处理过程处于关中断状态。

软件方面：ASYYE 将 32 号中断包装成 `_EVENT_IRQ_TIME`；nanos-lite 收到后进行 `schedule(r)`，而系统调用路径只处理系统调用本身，不再需要多 `schedule()` 一次。为了用户进程运行时能响应时钟中断，`_umake()` 构造的初始现场设置了 `EFLAGS` 的 `IF` 位。

实际的代码修改过程如下。

- 在 `reg.h` 的 `CPU_state` 中加入了 `bool INTR`，并在 `monitor.c` 复位初始化为 `false`。
- 在 `intr.c` 中实现 `dev_raise_intr()` 置一 `cpu.INTR` 位；同时在 `intr.c` 的 `raise_intr()` 保存旧 `EFLAGS` 后清 `cpu.IF`：

```

1  cpu.IF = 0;
2  val = cpu.cs;
3  rtl_push(&val);
4  val = ret_addr;
5  rtl_push(&val);

```

- 在 `exec.c` 定义 `#define TIMER_IRQ 32`，并在 `exec.c` 的 `exec_wrapper()` 末尾轮询 `cpu.INTR && cpu.IF`，响应时钟中断后再 `update_eip()` 跳到 `IDT` 中的处理入口：

```

1  if (cpu.INTR && cpu.IF) {
2      cpu.INTR = false;
3      raise_intr(TIMER_IRQ, cpu.eip);
4      update_eip();

```

```
5 }
```

- 在 trap.S 增加 vectimer, 将 32 号中断送入统一 asm_trap 处理流程:

```
1 .globl vectimer; vectimer: pushl $0; pushl $32; jmp asm_trap
```

- 在 asye.c 将 `tf->irq == 32` 包装成 `_EVENT_IRQ_TIME`, 并在 asye.c 注册到 `idt[32]` 表项中。
- 在 irq.c 中保留系统调用只 `do_syscall(r)`, 不再系统调用后再调度; 而收到 `_EVENT_IRQ_TIME` 后则调用 `schedule(r)` 进行调度:

```
1 case _EVENT_IRQ_TIME:
2     /* [PA 4] 时钟中断抢回控制权, 在中断返回前进行进程调度。 */
3     if (!timer_logged) {
4         Log("收到时钟中断");
5         timer_logged = true;
6     }
```

这里加了 `timer_logged`, 避免时钟频率过高把 Log 刷爆。

- 在 pte.c 的 `_umake()` 中设置 `tf->eflags = 0x2 | FL_IF`, 保证用户程序通过 `iret` 开始运行后处于开中断状态。

以上代码实现完备后, 再次 `make update` 并 `make run`, 运行结果如下:

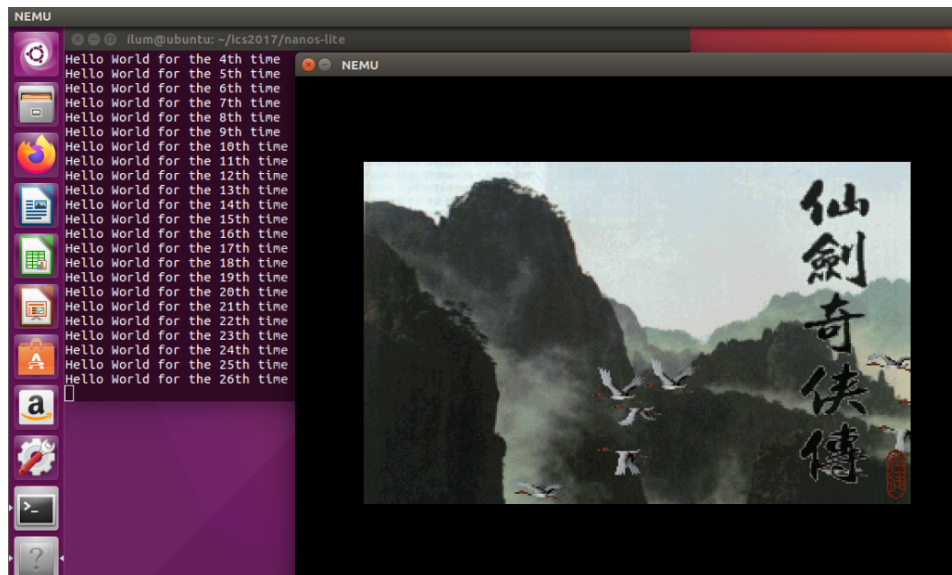
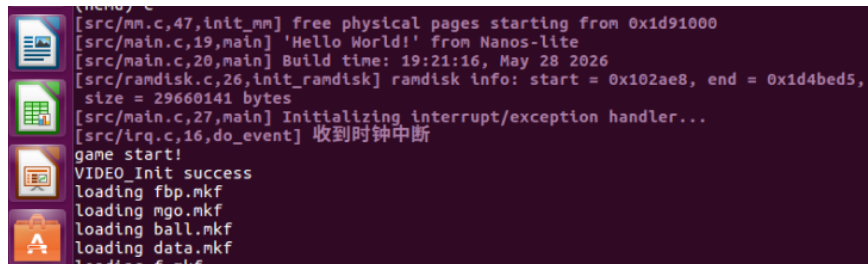


图 11: 时钟中断调度的 PAL 和 hello 同时运行



```

[src/mm.c,47,init_mm] free physical pages starting from 0x1d91000
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 19:21:16, May 28 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x102ae8, end = 0x1d4bed5,
size = 29660141 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/irq.c,16,do_event] 收到时钟中断
game start!
VIDEO_Init success
loading fbp.mkf
loading mgo.mkf
loading ball.mkf
loading data.mkf

```

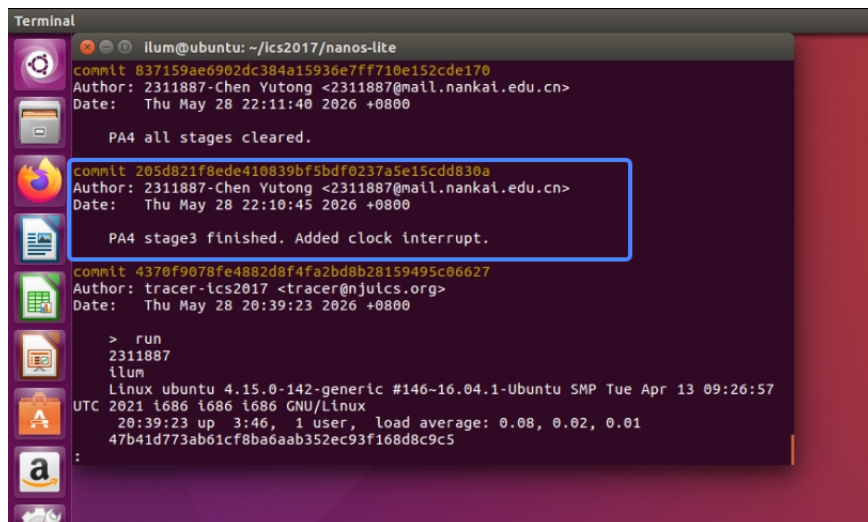
图 12: 时钟中断 Log 输出

可以看到，PAL 和 hello 按照设定的时钟轮转同时运行，PAL 流畅运行、hello 只是偶尔输出。并且终端输出了“收到时钟中断”的 Log，说明时钟中断的链路是通的，上述代码实现正确。

(十一) CHECKPOINT: PA4 stage3

——至此 PA4 stage3 完成。

手动 git 本地记录如下：



```

Terminal
illum@ubuntu: ~/ics2017/nanos-lite
commit 837159ae6902dc384a15936e7ff710e152cde170
Author: 2311887-Chen Yutong <2311887@mail.nankai.edu.cn>
Date: Thu May 28 22:11:40 2026 +0800

    PA4 all stages cleared.

commit 205d821f8ede410839bf5bdf0237a5e15cdd830a
Author: 2311887-Chen Yutong <2311887@mail.nankai.edu.cn>
Date: Thu May 28 22:10:45 2026 +0800

    PA4 stage3 finished. Added clock interrupt.

commit 4370f9078fe4882d8f4fa2bd8b28159495c06627
Author: tracer-ics2017 <tracer@njuics.org>
Date: Thu May 28 20:39:23 2026 +0800

    > run
    2311887
    illum
    Linux ubuntu 4.15.0-142-generic #146~16.04.1-Ubuntu SMP Tue Apr 13 09:26:57
    UTC 2021 i686 i686 i686 GNU/Linux
    20:39:23 up 3:46, 1 user, load average: 0.08, 0.02, 0.01
    47b41d773ab61cf8ba6aab352ec93f168d8c9c5
    :

```

图 13: 手动 git 本地记录

至此 PA 4 代码实现全部完成

四、 实验环境说明

由于整个课程的实验部分没有要求提交 PA0 / 环境配置一节的报告，此处对实验环境的搭建作简单说明。

(一) 虚拟机配置

采用 VMware 15.5.0 + Ubuntu 16.04 ([ubuntu-16.04.4-desktop-i386.iso](#)) 有可视化界面的虚拟机环境。

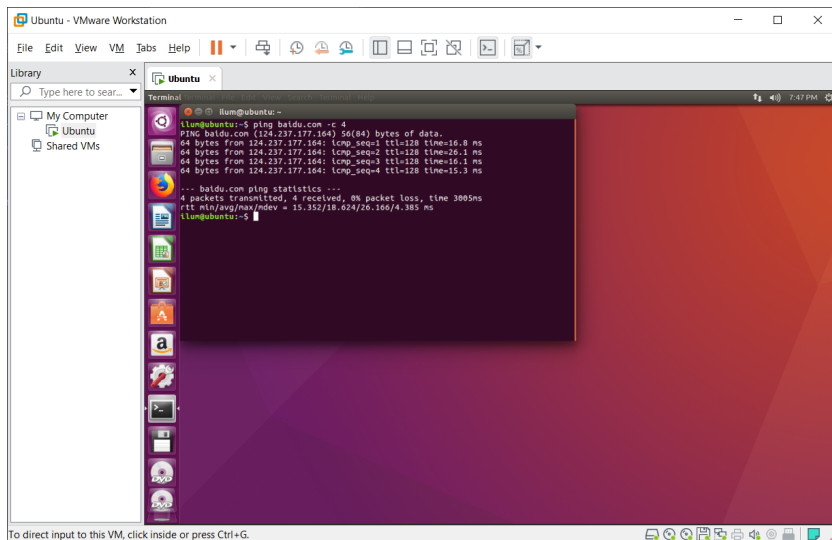


图 14: VMWare + Ubuntu

网络连接无问题。

按照指导书要求，使用 `sudo apt-get install` 命令下载并安装相应的资源工具包。

(二) IDE 配置

此前编程习惯为 vscode 的 IDE 环境，故尝试将此 Ubuntu 虚拟机连接到 vscode。

此前尝试 vscode 的 SSH 功能、VMware 自带的共享文件夹，均失败。

最终使用 SSHFS 的方案，把 Ubuntu 的目录通过 SSHFS 作为一个附加卷挂载到宿主 Windows。安装 WinFsp 和 SSHFS-Win 工具，而后只需要在 Windows cmd 输入命令 `net use X: \\sshfs\[username]@[ip.addr]\[password] /user:[username]`（方括号需要替换为相应内容）在宿主机的文件系统根目录形成一个新的卷轴“X:”，其中内容就是对 Ubuntu 虚拟机指定文件夹的挂载。在 X 盘内右键用 vscode 打开即可用 IDE 环境来编辑代码，同步生效于 Ubuntu 虚拟机。

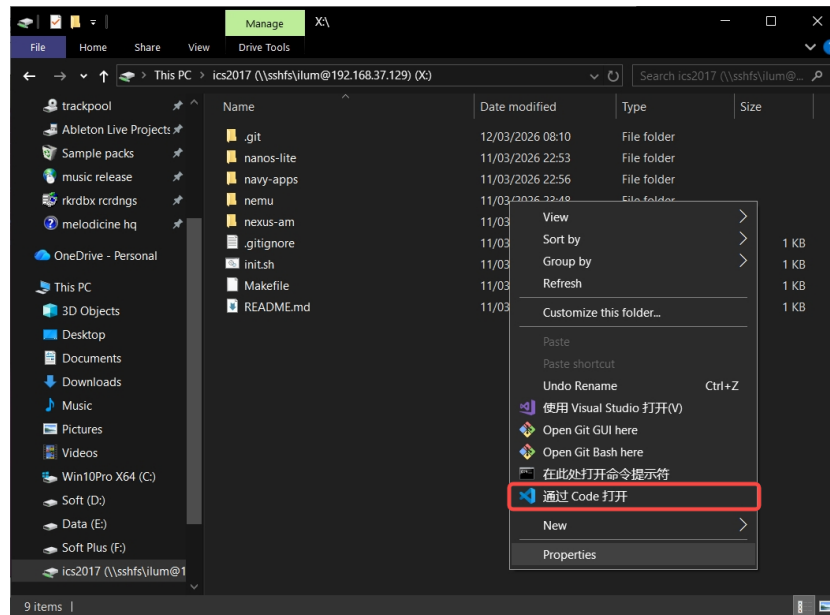


图 15: 本地 vscode 编写代码

五、 总结

本次 PA4 的主线是把 NEMU、AM 和 Nanos-lite 串成真正支持用户进程的系统：先补充分页相关寄存器、地址翻译和跨页读写，再在 AM/Nanos-lite 中建立用户地址空间、加载程序并维护内核映射；随后通过内核自陷、上下文切换和调度器运行 PAL 与 hello，最后加入时钟中断，让分时不再依赖系统调用。实现过程中最容易出问题的是页表地址必须使用物理地址、用户页目录必须复制内核映射，以及 trap frame 必须随栈保存现场；这些问题也帮助我更清楚地理解了地址空间、异常入口和进程切换之间的关系。

PA4 实验代码已同步备份至 [GitHub 仓库](#)。